B.TECH CSE | PYTHON PROGRAMMING

Python Programming Study Notes

Clear concepts, code examples, and exam practice

A module-wise guide to the syllabus supplied for these notes

Simple explanation first. Formal definition next. Then trace the code.

Follow each module roadmap; write outputs by hand before checking.

STUDENT EDITION | 116 PAGES | 3 MODULES

A clear route through the syllabus

How to study each topic

- Read the simple definition first, then compare it with the formal definition.
- Type or trace the example yourself; predict the output before reading it.
- Use the common-mistake and viva boxes to test whether the idea is clear.
- Follow the exact module and topic sequence shown in the roadmaps.

What these notes include

- All 48 listed syllabus headings appear in their module order.
- Each topic has a simple explanation, technical definition, notation, code, and exam cues.
- Revision definitions, syntax reference, graded practice, and answer cues are at the end.
- The two pages marked Additional / Outside Syllabus contain optional coding habits only.

A reliable way to trace code

- Make a small table for changing variables, loop turns, or object attributes.
- Keep strings in quotes and distinguish printed output from a returned value.
- For errors, identify the statement, exception type, and recovery path.
- For classes, track each object's attributes separately.

QUICK RECALL

Python becomes easier when you predict one small step at a time.

MODULE-I - what to study and in what order

Module goal

Build Python foundations, then control program flow and pass flexible inputs to functions/scripts.

Syllabus map - exact topic order

- Python Basics: Operators; Precedence and Associativity; Data Types; Indentation; Comments; Reading Input; Print Output; Type Conversions; The type() Function and Is Operator; Dynamic and Strongly Typed Language.
- Control Flow Statements: The if Decision Control Flow Statement; The if...else Decision Control Flow Statement; The if...elif...else Decision Control Statement; Nested if Statement; The while Loop; The for Loop; The continue and break Statements; Catching Exceptions Using try and except Statement.
- Function Arguments and Command Line Arguments: Default Parameters; Keyword Arguments; *args and **kwargs; Command Line Arguments.

Recommended study order

- Start with operators, types, input/output, and the rules that shape expressions.
- Move through decisions, loops, and exception handling; trace each branch and iteration.
- Finish with defaults, keyword/packed arguments, and validated command-line input.

QUICK RECALL

Before moving on, trace a nested decision, a loop, and a function call with arguments.

TOPIC 01 / 48 | Python Basics CONCEPT AND DEFINITIONS

Operators

Simple definition

Operators tell Python to perform an action on values, such as add numbers, compare values, or combine conditions.

Formal / technical definition

An operator is a token that applies an operation to one or more operands and produces a result. Python includes arithmetic, comparison, assignment, logical, identity, membership, and bitwise operators.

Notation and how to read it

+, -, *, /, //, %, ** | ==, !=, <, >= | and, or, not | is, in | =, +=

How it works

- State the rule in the topic's formal terminology.
- Apply it to the example one step at a time.
- Check the result against the key distinction or limitation.

Quick map / trace / syntax

```
a + b: arithmetic
a == b: comparison
a and b: logical
x in items: membership
x is y: identity
```

QUICK RECALL

Choose the operator from the intended meaning; do not use `is` as a replacement for ordinary value comparison.

TOPIC 01 / 48 | Python Basics

CODE EXAMPLE AND EXAM HELP

Operators

Worked code example and output

```
Code:
a = 7
b = 3
print(a + b, a // b, a % b)
print(a > b and b != 0)
Output:
10 2 1
True
```

Useful distinction / limitation

`/` returns a floating-point quotient; `//` floors the quotient. `is` checks object identity, while `==` checks value equality.

Common mistakes to avoid

- State the topic's rule before applying it.
- Trace the data or control flow carefully; check the final value and type.

Quick viva and follow-up cues

- Core answer: Choose the operator from the intended meaning; do not use `is` as a replacement for ordinary value comparison.
- Likely follow-up: `/` returns a floating-point quotient; `//` floors the quotient. `is` checks object identity, while `==` checks value equality.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Choose the operator from the intended meaning; do not use `is` as a replacement for ordinary value comparison.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 02 / 48 | Python Basics CONCEPT AND DEFINITIONS

Precedence and Associativity

Simple definition

Precedence decides which operator is grouped first. Associativity decides how operators of the same precedence group when parentheses are absent.

Formal / technical definition

Python parses an expression according to a precedence table and associativity rules. Parentheses override the default grouping. Exponentiation is right-associative; most arithmetic operators at the same level group left-to-right.

Notation and how to read it

2 + 3 * 4 = 2 + (3 * 4)

10 - 2 - 1 = (10 - 2) - 1

2 ** 3 ** 2 = 2 ** (3 ** 2)

How it works

- State the rule in the topic's formal terminology.
- Apply it to the example one step at a time.
- Check the result against the key distinction or limitation.

Quick map / trace / syntax

```
Parentheses -> exponentiation -> unary -> multiply/divide -> add/subtract -> comparisons -> not -> and -> or
```

QUICK RECALL

Do not assume every operator groups left-to-right; know the exponentiation exception and use parentheses for clarity.

TOPIC 02 / 48 | Python Basics

CODE EXAMPLE AND EXAM HELP

Precedence and Associativity

Worked code example and output

```
Code:
print(2 + 3 * 4)
print((2 + 3) * 4)
print(10 - 2 - 1)
print(2 ** 3 ** 2)
Output:
14
20
7
512
```

Useful distinction / limitation

When an expression is important to a reader, use parentheses even if precedence already makes the result unambiguous.

Common mistakes to avoid

- Do not guess a long expression's grouping; use the precedence rules or parentheses.
- Exponentiation groups right-to-left; subtraction groups left-to-right.

Quick viva and follow-up cues

- Core answer: Do not assume every operator groups left-to-right; know the exponentiation exception and use parentheses for clarity.
- Likely follow-up: When an expression is important to a reader, use parentheses even if precedence already makes the result unambiguous.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Do not assume every operator groups left-to-right; know the exponentiation exception and use parentheses for clarity.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 03 / 48 | Python Basics

CONCEPT AND DEFINITIONS

Data Types

Simple definition

A data type tells Python what kind of value is stored and which operations make sense for it.

Formal / technical definition

Python objects have types. Common built-in types include `int`, `float`, `complex`, `bool`, `str`, `list`, `tuple`, `dict`, `set`, and `NoneType`. Types determine supported operations and mutability behavior.

Notation and how to read it

12 -> int | 3.5 -> float | True -> bool | "OS" -> str | [1, 2] -> list

How it works

- State the rule in the topic's formal terminology.
- Apply it to the example one step at a time.
- Check the result against the key distinction or limitation.

Quick map / trace / syntax

```
Value -> object -> type controls valid operations
Mutable examples: list, dict, set
Immutable examples: int, float, bool, str, tuple
```

QUICK RECALL

Distinguish an object's type from the variable name that currently refers to it.

TOPIC 03 / 48 | Python Basics

CODE EXAMPLE AND EXAM HELP

Data Types

Worked code example and output

```
Code:
values = [12, 3.5, True, "OS", [1, 2]]
for value in values:
print(type(value).__name__)
Output:
int
float
bool
str
list
```

Useful distinction / limitation

A variable name refers to an object; assigning a new object to the name does not change the old object's type.

Common mistakes to avoid

- State the topic's rule before applying it.
- Trace the data or control flow carefully; check the final value and type.

Quick viva and follow-up cues

- Core answer: Distinguish an object's type from the variable name that currently refers to it.
- Likely follow-up: A variable name refers to an object; assigning a new object to the name does not change the old object's type.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Distinguish an object's type from the variable name that currently refers to it.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 04 / 48 | Python Basics

CONCEPT AND DEFINITIONS

Indentation

Simple definition

Indentation is the spaces at the start of a line. Python uses it to show which statements belong inside a block.

Formal / technical definition

A suite is a group of statements associated with a clause such as `if`, `for`, `while`, `def`, or `class`. A colon begins the suite; consistent indentation defines its extent.

Notation and how to read it

header + colon:

indented suite

next line at earlier level = outside that suite

How it works

- State the rule in the topic's formal terminology.
- Apply it to the example one step at a time.
- Check the result against the key distinction or limitation.

Quick map / trace / syntax

```
if condition:
statement in block
another statement
statement after block
```

QUICK RECALL

A line's indentation is part of Python syntax; align statements that belong to the same block.

TOPIC 04 / 48 | Python Basics

CODE EXAMPLE AND EXAM HELP

Indentation

Worked code example and output

```
Code:
score = 82
if score >= 50:
print("Pass")
print("Keep going")
print("Done")
Output:
Pass
Keep going
Done
```

Useful distinction / limitation

Use consistent spaces (commonly four per level). Mixing tabs and spaces can cause errors or confusing layouts.

Common mistakes to avoid

- State the topic's rule before applying it.
- Trace the data or control flow carefully; check the final value and type.

Quick viva and follow-up cues

- Core answer: A line's indentation is part of Python syntax; align statements that belong to the same block.
- Likely follow-up: Use consistent spaces (commonly four per level). Mixing tabs and spaces can cause errors or confusing layouts.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: A line's indentation is part of Python syntax; align statements that belong to the same block.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 05 / 48 | Python Basics CONCEPT AND DEFINITIONS

Comments

Simple definition

A comment is a note for people reading code. Python ignores a `#` comment when it runs the program.

Formal / technical definition

A comment begins with `#` outside a string literal and continues to the end of that physical line. Triple-quoted strings are string literals; when placed as the first statement in a module, class, or function, they can serve as docstrings.

Notation and how to read it

one-line comment

"""Docstring describing a module/function."""

How it works

- State the rule in the topic's formal terminology.
- Apply it to the example one step at a time.
- Check the result against the key distinction or limitation.

Quick map / trace / syntax

```
Code line -> interpreter executes
# explanatory text -> interpreter skips it
Docstring -> stored documentation string
```

QUICK RECALL

Do not confuse a docstring with a comment: both help readers, but a docstring is a string object attached to code documentation.

TOPIC 05 / 48 | Python Basics

CODE EXAMPLE AND EXAM HELP

Comments

Worked code example and output

```
Code:
# Convert Celsius to Fahrenheit
celsius = 20
fahrenheit = celsius * 9 / 5 + 32
print(fahrenheit)
Output:
68.0
```

Useful distinction / limitation

Comments should explain intent or a non-obvious decision, not merely repeat the code. A docstring is accessible at runtime; a comment is not.

Common mistakes to avoid

- State the topic's rule before applying it.
- Trace the data or control flow carefully; check the final value and type.

Quick viva and follow-up cues

- Core answer: Do not confuse a docstring with a comment: both help readers, but a docstring is a string object attached to code documentation.
- Likely follow-up: Comments should explain intent or a non-obvious decision, not merely repeat the code. A docstring is accessible at runtime; a comment is not.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Do not confuse a docstring with a comment: both help readers, but a docstring is a string object attached to code documentation.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 06 / 48 | Python Basics

CONCEPT AND DEFINITIONS

Reading Input

Simple definition

`input()` pauses for a line typed by the user and gives the program that line as text.

Formal / technical definition

The built-in `input(prompt)` writes an optional prompt to standard output, reads a line from standard input, removes the trailing newline, and returns a `str`.

Notation and how to read it

text = input(prompt)

input() always returns str

How it works

- State the rule in the topic's formal terminology.
- Apply it to the example one step at a time.
- Check the result against the key distinction or limitation.

Quick map / trace / syntax

```
keyboard text -> input() -> string value -> optional conversion -> program logic
```

QUICK RECALL

Never assume `input()` returns an integer just because the user typed digits.

TOPIC 06 / 48 | Python Basics

CODE EXAMPLE AND EXAM HELP

Reading Input

Worked code example and output

```
Code:
name = input("Name: " )
print("Hello,", name)
Sample input: Asha
Output:
Name: Asha
Hello, Asha
```

Useful distinction / limitation

Convert only when numeric operations are needed, and handle invalid text when user input may be malformed.

Common mistakes to avoid

- Do not assume input() returns a number; it returns a string.
- Convert and validate before arithmetic.

Quick viva and follow-up cues

- Core answer: Never assume `input()` returns an integer just because the user typed digits.
- Likely follow-up: Convert only when numeric operations are needed, and handle invalid text when user input may be malformed.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Never assume `input()` returns an integer just because the user typed digits.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 07 / 48 | Python Basics

CONCEPT AND DEFINITIONS

Print Output

Simple definition

`print()` displays values. It can print several values, choose a separator, and choose what appears at the end.

Formal / technical definition

The built-in `print(*objects, sep=' ', end='\n', file=None, flush=False)` converts objects to text and writes them to a text stream, normally standard output.

Notation and how to read it

sep controls gaps between objects; end controls the final text (default newline).

How it works

- State the rule in the topic's formal terminology.
- Apply it to the example one step at a time.
- Check the result against the key distinction or limitation.

Quick map / trace / syntax

```
print(value1, value2, sep=" / ", end="!\n")
```

QUICK RECALL

The default `end` is a newline. If it is changed, the next print may continue on the same line.

TOPIC 07 / 48 | Python Basics

CODE EXAMPLE AND EXAM HELP

Print Output

Worked code example and output

```
Code:
print("A", 5, True, sep=" | ")
print("Loading", end="...")
print("done")
Output:
A | 5 | True
Loading...done
```

Useful distinction / limitation

Use f-strings for readable labels and values; use `sep`/`end` when output layout requires them.

Common mistakes to avoid

- Remember that sep is between objects and end follows the last object.
- Printing a value is different from returning it from a function.

Quick viva and follow-up cues

- Core answer: The default `end` is a newline. If it is changed, the next print may continue on the same line.
- Likely follow-up: Use f-strings for readable labels and values; use `sep`/`end` when output layout requires them.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: The default `end` is a newline. If it is changed, the next print may continue on the same line.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 08 / 48 | Python Basics CONCEPT AND DEFINITIONS

Type Conversions

Simple definition

Type conversion changes a value to another type. Some conversions are explicit, such as `int('12')`; some happen automatically in mixed numeric expressions.

Formal / technical definition

A conversion constructs or derives an object of a target type. Explicit constructors include `int`, `float`, `str`, `bool`, `list`, and `tuple`; a conversion can fail when the source representation is invalid.

Notation and how to read it

int('18') -> 18
float('2.5') -> 2.5
str(18) -> '18'
invalid numeric text -> ValueError

How it works

- State the rule in the topic's formal terminology.
- Apply it to the example one step at a time.
- Check the result against the key distinction or limitation.

Quick map / trace / syntax

```
input string -> conversion -> typed value -> calculation
```

QUICK RECALL

Do not mix representation and value: `'18'` is text until converted to a number.

TOPIC 08 / 48 | Python Basics

CODE EXAMPLE AND EXAM HELP

Type Conversions

Worked code example and output

```
Code:
text = "18"
age = int(text)
price = float(2)
print(age + 1, price)
Output:
19 2.0
```

Useful distinction / limitation

`bool('False')` is `True` because every non-empty string is truthy. Use explicit parsing when text encodes yes/no.

Common mistakes to avoid

- State the topic's rule before applying it.
- Trace the data or control flow carefully; check the final value and type.

Quick viva and follow-up cues

- Core answer: Do not mix representation and value: `'18'` is text until converted to a number.
- Likely follow-up: `bool('False')` is `True` because every non-empty string is truthy. Use explicit parsing when text encodes yes/no.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Do not mix representation and value: `'18'` is text until converted to a number.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 09 / 48 | Python Basics CONCEPT AND DEFINITIONS

The type() Function and Is Operator

Simple definition

`type()` reports an object's type. `is` asks whether two names refer to the very same object; it does not ask whether their values match.

Formal / technical definition

`type(object)` returns the object's class/type. The identity operator `is` is true when both operands refer to the same object. Use `==` for value equality; use `is None` for the conventional None check.

Notation and how to read it

a == b -> equal values

a is b -> identical object

type(a) -> object's class

How it works

- State the rule in the topic's formal terminology.
- Apply it to the example one step at a time.
- Check the result against the key distinction or limitation.

Quick map / trace / syntax

```
a ----> list object <---- b
c ----> a different list with equal contents
```

QUICK RECALL

Use `is None` for None checks; use `==` for strings, numbers, and collection contents.

TOPIC 09 / 48 | Python Basics

CODE EXAMPLE AND EXAM HELP

The type() Function and Is Operator

Worked code example and output

```
Code:
a = [1, 2]
b = a
c = [1, 2]
print(type(a).__name__, a == c, a is b, a is c)
print(a is None)
Output:
list True True False
False
```

Useful distinction / limitation

Object interning can make identity of some immutable values look surprising. Never rely on it for value comparisons.

Common mistakes to avoid

- Do not use is to compare ordinary values or collection contents.
- Use == for equality and is None for the None singleton.

Quick viva and follow-up cues

- Core answer: Use `is None` for None checks; use `==` for strings, numbers, and collection contents.
- Likely follow-up: Object interning can make identity of some immutable values look surprising. Never rely on it for value comparisons.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Use `is None` for None checks; use `==` for strings, numbers, and collection contents.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 10 / 48 | Python Basics CONCEPT AND DEFINITIONS

Dynamic and Strongly Typed Language

Simple definition

Python is dynamically typed: a name can refer to objects of different types at different times. It is strongly typed: Python does not silently combine incompatible values in many operations.

Formal / technical definition

Type binding is associated with runtime objects rather than fixed variable declarations. Operations still enforce type rules; incompatible operands raise exceptions unless an explicit conversion or defined coercion applies.

Notation and how to read it

dynamic: name -> object at runtime

strong: incompatible operation -> error (unless explicitly converted)

How it works

- State the rule in the topic's formal terminology.
- Apply it to the example one step at a time.
- Check the result against the key distinction or limitation.

Quick map / trace / syntax

```
name x -> int object
name x -> later, str object
5 + '2' -> TypeError
```

QUICK RECALL

Do not describe Python as untyped; distinguish when types are bound from how strictly operations are checked.

TOPIC 10 / 48 | Python Basics

CODE EXAMPLE AND EXAM HELP

Dynamic and Strongly Typed Language

Worked code example and output

```
Code:
value = 10
value = "ten"
print(value.upper())
try:
print(5 + "2")
except TypeError as error:
print(type(error).__name__)
Output:
TEN
TypeError
```

Useful distinction / limitation

Dynamic typing is not the same as no typing: every object has a type, and Python checks whether an operation is valid.

Common mistakes to avoid

- Dynamic typing does not mean untyped.
- Strong typing means incompatible values do not silently become compatible.

Quick viva and follow-up cues

- Core answer: Do not describe Python as untyped; distinguish when types are bound from how strictly operations are checked.
- Likely follow-up: Dynamic typing is not the same as no typing: every object has a type, and Python checks whether an operation is valid.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Do not describe Python as untyped; distinguish when types are bound from how strictly operations are checked.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 11 / 48 | Control Flow Statements CONCEPT AND DEFINITIONS

The if Decision Control Flow Statement

Simple definition

An `if` statement runs its indented block only when its condition is true.

Formal / technical definition

An `if` statement evaluates a condition using truth-value testing. If true, Python executes the associated suite; otherwise it skips it and continues after the suite.

Notation and how to read it

if condition:

suite

How it works

- Evaluate the condition or conditions from left to right.
- Trace which indented block runs and which blocks are skipped.
- Check boundary values and confirm that only the intended path executes.

Quick map / trace / syntax

```
condition true -> run block
condition false -> skip block
```

QUICK RECALL

The colon and indentation are required; the body is not controlled by visual spacing alone but by Python's indentation rules.

TOPIC 11 / 48 | Control Flow Statements CODE EXAMPLE AND EXAM HELP

The if Decision Control Flow Statement

Worked code example and output

```
Code:
marks = 72
if marks >= 40:
print("Pass")
print("Result checked")
Output:
Pass
Result checked
```

Useful distinction / limitation

Values such as empty strings, zero, empty collections, and `None` are falsey; non-empty strings and most objects are truthy.

Common mistakes to avoid

- State the topic's rule before applying it.
- Trace the data or control flow carefully; check the final value and type.

Quick viva and follow-up cues

- Core answer: The colon and indentation are required; the body is not controlled by visual spacing alone but by Python's indentation rules.
- Likely follow-up: Values such as empty strings, zero, empty collections, and `None` are falsey; non-empty strings and most objects are truthy.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: The colon and indentation are required; the body is not controlled by visual spacing alone but by Python's indentation rules.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 12 / 48 | Control Flow Statements CONCEPT AND DEFINITIONS

The if...else Decision Control Flow Statement

Simple definition

`if...else` chooses one of two paths: one block runs when the condition is true, the other when it is false.

Formal / technical definition

The `else` suite is paired with an `if` statement and executes only when that `if` condition is false. Exactly one of the two suites is selected.

Notation and how to read it

if condition: true_suite

else: false_suite

How it works

- Evaluate the condition or conditions from left to right.
- Trace which indented block runs and which blocks are skipped.
- Check boundary values and confirm that only the intended path executes.

Quick map / trace / syntax

```
condition?
yes -> if suite
no -> else suite
```

QUICK RECALL

Avoid duplicating the condition in the `else`; it already means the condition was false.

TOPIC 12 / 48 | Control Flow Statements CODE EXAMPLE AND EXAM HELP

The if...else Decision Control Flow Statement

Worked code example and output

```
Code:
number = 9
if number % 2 == 0:
print("Even")
else:
print("Odd")
Output:
Odd
```

Useful distinction / limitation

The `else` belongs to the nearest unmatched `if` at the same indentation level.

Common mistakes to avoid

- State the topic's rule before applying it.
- Trace the data or control flow carefully; check the final value and type.

Quick viva and follow-up cues

- Core answer: Avoid duplicating the condition in the `else`; it already means the condition was false.
- Likely follow-up: The `else` belongs to the nearest unmatched `if` at the same indentation level.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Avoid duplicating the condition in the `else`; it already means the condition was false.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 13 / 48 | Control Flow Statements CONCEPT AND DEFINITIONS

The if...elif...else Decision Control Statement

Simple definition

Use `elif` when there are several mutually exclusive cases. Python checks conditions from top to bottom and runs the first matching block.

Formal / technical definition

An `if` statement may contain zero or more `elif` clauses and an optional `else`. Conditions are evaluated sequentially; after one is true and its suite runs, remaining clauses are skipped.

Notation and how to read it

if C1 -> S1

elif C2 -> S2

else -> fallback

How it works

- Evaluate the condition or conditions from left to right.
- Trace which indented block runs and which blocks are skipped.
- Check boundary values and confirm that only the intended path executes.

Quick map / trace / syntax

```
check C1 -> if false, check C2 -> if false, use else
```

QUICK RECALL

Only the first true branch runs. Test boundary values such as 74, 75, 89, and 90.

TOPIC 13 / 48 | Control Flow Statements CODE EXAMPLE AND EXAM HELP

The if...elif...else Decision Control Statement

Worked code example and output

```
Code:
score = 84
if score >= 90:
grade = "A"
elif score >= 75:
grade = "B"
else:
grade = "C"
print(grade)
Output:
B
```

Useful distinction / limitation

Put more specific or higher-threshold tests first; a broad earlier condition can make later branches unreachable.

Common mistakes to avoid

- State the topic's rule before applying it.
- Trace the data or control flow carefully; check the final value and type.

Quick viva and follow-up cues

- Core answer: Only the first true branch runs. Test boundary values such as 74, 75, 89, and 90.
- Likely follow-up: Put more specific or higher-threshold tests first; a broad earlier condition can make later branches unreachable.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Only the first true branch runs. Test boundary values such as 74, 75, 89, and 90.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 14 / 48 | Control Flow Statements CONCEPT AND DEFINITIONS

Nested if Statement

Simple definition

A nested `if` is an `if` inside another `if`. The inner decision is reached only if execution enters the outer block.

Formal / technical definition

Nested selection composes conditional suites. Each nested condition is evaluated only along the execution path where its enclosing condition's suite is entered.

Notation and how to read it

outer condition must be true before the inner condition is checked

How it works

- Evaluate the condition or conditions from left to right.
- Trace which indented block runs and which blocks are skipped.
- Check boundary values and confirm that only the intended path executes.

Quick map / trace / syntax

```
age check
true -> ID check
true -> allow
false -> ask for ID
false -> deny
```

QUICK RECALL

Indent each level consistently and identify which `else` pairs with which `if`.

TOPIC 14 / 48 | Control Flow Statements

CODE EXAMPLE AND EXAM HELP

Nested if Statement

Worked code example and output

```
Code:
age = 20
has_id = True
if age >= 18:
if has_id:
print("Entry allowed")
else:
print("Show ID")
else:
print("Too young")
Output:
Entry allowed
```

Useful distinction / limitation

For many exclusive choices, `if...elif...else` may be clearer than deeply nested blocks.

Common mistakes to avoid

- State the topic's rule before applying it.
- Trace the data or control flow carefully; check the final value and type.

Quick viva and follow-up cues

- Core answer: Indent each level consistently and identify which `else` pairs with which `if`.
- Likely follow-up: For many exclusive choices, `if...elif...else` may be clearer than deeply nested blocks.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Indent each level consistently and identify which `else` pairs with which `if`.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 15 / 48 | Control Flow Statements CONCEPT AND DEFINITIONS

The while Loop

Simple definition

A `while` loop repeats a block as long as its condition remains true. The loop needs a path that eventually makes the condition false or exits.

Formal / technical definition

A `while` statement repeatedly evaluates its expression before each iteration. A false result ends the loop; `break`, exception, or function return can also leave it.

Notation and how to read it

initialize -> test -> body -> update -> test again

How it works

- Initialize the loop-control value.
- Test the condition before each iteration; run the suite only when true.
- Update state or exit so the loop can finish.

Quick map / trace / syntax

```
condition true -> body -> update
condition false -> continue after loop
```

QUICK RECALL

Trace the loop variable and condition after each iteration, including the final false test.

TOPIC 15 / 48 | Control Flow Statements

CODE EXAMPLE AND EXAM HELP

The while Loop

Worked code example and output

```
Code:
count = 1
while count <= 3:
print(count)
count += 1
Output:
1
2
3
```

Useful distinction / limitation

A missing update can create an infinite loop. For input-controlled loops, ensure every path either changes state or exits.

Common mistakes to avoid

- Check that the loop changes state or exits; otherwise it may never terminate.
- The condition is tested before each iteration.

Quick viva and follow-up cues

- Core answer: Trace the loop variable and condition after each iteration, including the final false test.
- Likely follow-up: A missing update can create an infinite loop. For input-controlled loops, ensure every path either changes state or exits.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Trace the loop variable and condition after each iteration, including the final false test.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 16 / 48 | Control Flow Statements CONCEPT AND DEFINITIONS

The for Loop

Simple definition

A `for` loop takes items from an iterable one at a time and runs the block for each item.

Formal / technical definition

A `for` statement obtains an iterator from an iterable and assigns each yielded value to the loop target until iteration is exhausted or control exits early.

Notation and how to read it

range(start, stop, step) excludes stop; an iterable supplies successive values

How it works

- Identify the iterable and the values it yields.
- Trace one value per iteration and update any accumulator.
- Stop when the iterator is exhausted or a control statement exits early.

Quick map / trace / syntax

```
iterable -> next item -> loop body -> next item -> exhausted
```

QUICK RECALL

Know whether the iterable yields values or indexes; use `enumerate(items)` when both are needed.

TOPIC 16 / 48 | Control Flow Statements

CODE EXAMPLE AND EXAM HELP

The for Loop

Worked code example and output

```
Code:
for number in range(2, 8, 2):
print(number)
Output:
2
4
6
```

Useful distinction / limitation

`range(2, 8, 2)` stops before 8. To enumerate both endpoints, adjust the stop value when appropriate.

Common mistakes to avoid

- range stop is excluded.
- Do not change a list's length while relying on its iterator positions without a clear plan.

Quick viva and follow-up cues

- Core answer: Know whether the iterable yields values or indexes; use `enumerate(items)` when both are needed.
- Likely follow-up: `range(2, 8, 2)` stops before 8. To enumerate both endpoints, adjust the stop value when appropriate.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Know whether the iterable yields values or indexes; use `enumerate(items)` when both are needed.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 17 / 48 | Control Flow Statements CONCEPT AND DEFINITIONS

The continue and break Statements

Simple definition

`continue` skips the rest of the current loop turn. `break` exits the nearest enclosing loop completely.

Formal / technical definition

These control-transfer statements alter the innermost loop: `continue` begins the next iteration, while `break` terminates the loop and transfers control to its `else`/following statement as applicable.

Notation and how to read it

continue -> next iteration

break -> leave nearest loop

How it works

- Locate the innermost loop containing the statement.
- For continue, skip to the next iteration; for break, exit that loop.
- Trace the remaining statements to show exactly what runs.

Quick map / trace / syntax

```
loop -> condition -> continue? skip body remainder
break? exit loop
```

QUICK RECALL

Trace which statements are skipped and which loop the `break` targets.

TOPIC 17 / 48 | Control Flow Statements CODE EXAMPLE AND EXAM HELP

The continue and break Statements

Worked code example and output

```
Code:
for n in range(1, 6):
if n == 2:
continue
if n == 5:
break
print(n)
Output:
1
3
4
```

Useful distinction / limitation

With nested loops, `break` exits only the innermost loop. A flag or function may be needed to stop an outer loop too.

Common mistakes to avoid

- continue skips only the current iteration; break exits only the innermost loop.
- Check which later statements are skipped.

Quick viva and follow-up cues

- Core answer: Trace which statements are skipped and which loop the `break` targets.
- Likely follow-up: With nested loops, `break` exits only the innermost loop. A flag or function may be needed to stop an outer loop too.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Trace which statements are skipped and which loop the `break` targets.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 18 / 48 | Control Flow Statements CONCEPT AND DEFINITIONS

Catching Exceptions Using try and except Statement

Simple definition

`try` marks code that might fail. A matching `except` block handles a particular error so the program can respond clearly.

Formal / technical definition

When an exception is raised in a `try` suite, Python searches the associated handlers in order for a compatible exception type. A matching handler executes; otherwise the exception propagates.

Notation and how to read it

try: risky operation

except SpecificError: recovery action

How it works

- Place statements that may raise inside the try suite.
- Catch the narrow exception type that the program can handle.
- Continue with a safe result or show a clear recovery message.

Quick map / trace / syntax

```
success -> continue
matching exception -> handler
unmatched exception -> propagates
```

QUICK RECALL

Put only the risky operation in `try`; name the exception type and give the user a meaningful recovery path.

TOPIC 18 / 48 | Control Flow Statements CODE EXAMPLE AND EXAM HELP

Catching Exceptions Using try and except Statement

Worked code example and output

```
Code:
text = "abc"
try:
amount = int(text)
except ValueError:
amount = 0
print(amount)
Output:
0
```

Useful distinction / limitation

Catch the narrow exception you can handle. A bare `except` can hide programming errors and should be avoided in ordinary examples.

Common mistakes to avoid

- Do not catch every exception when only one expected error needs handling.
- Do not put unrelated code inside the try block.

Quick viva and follow-up cues

- Core answer: Put only the risky operation in `try`; name the exception type and give the user a meaningful recovery path.
- Likely follow-up: Catch the narrow exception you can handle. A bare `except` can hide programming errors and should be avoided in ordinary examples.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Put only the risky operation in `try`; name the exception type and give the user a meaningful recovery path.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

Default Parameters

Simple definition

A default parameter supplies a value when the caller leaves that argument out.

Formal / technical definition

A parameter with a default expression may be omitted in a call. Defaults are evaluated once when the function is defined, not afresh on every call.

Notation and how to read it

def f(required, optional=default): ...

How it works

- List required and optional parameters in the function definition.
- Call once with the argument omitted and once with it supplied.
- Check whether a mutable default could be shared between calls.

Quick map / trace / syntax

```
argument supplied -> use supplied value
argument omitted -> use default
```

QUICK RECALL

Required parameters normally come before defaulted parameters; understand that a default object can be shared across calls.

Default Parameters

Worked code example and output

```
Code:
def greet(name, greeting="Hello"):
print(greeting, name)
greet("Asha")
greet("Ravi", "Welcome")
Output:
Hello Asha
Welcome Ravi
```

Useful distinction / limitation

Avoid a mutable list/dictionary default when each call needs a fresh object; use `None` and create the object inside.

Common mistakes to avoid

- Defaults are evaluated once at function definition.
- Use None for a fresh mutable default on each call.

Quick viva and follow-up cues

- Core answer: Required parameters normally come before defaulted parameters; understand that a default object can be shared across calls.
- Likely follow-up: Avoid a mutable list/dictionary default when each call needs a fresh object; use `None` and create the object inside.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Required parameters normally come before defaulted parameters; understand that a default object can be shared across calls.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 20 / 48 | Function Arguments and Command Line Arguments CONCEPT AND DEFINITIONS

Keyword Arguments

Simple definition

A keyword argument names the parameter in the call, making the purpose clear and allowing arguments to be written in a different order.

Formal / technical definition

A keyword argument binds a value to a parameter by name. Positional arguments are matched by position; keyword arguments are matched by parameter name, subject to Python's calling rules.

Notation and how to read it

function(parameter=value, ...)

positional arguments must precede keyword arguments in a call

How it works

- Match each argument to its parameter name.
- Check for missing, unexpected, or duplicate parameter values.
- Call the function and verify the bound values inside the body.

Quick map / trace / syntax

```
name="Aman" -> name parameter
city="Delhi" -> city parameter
```

QUICK RECALL

Use keywords when they improve clarity, especially for optional parameters; keep parameter names accurate.

Keyword Arguments

Worked code example and output

```
Code:
def profile(name, city):
print(name, "lives in", city)
profile(city="Delhi", name="Aman")
Output:
Aman lives in Delhi
```

Useful distinction / limitation

A parameter cannot receive the same argument twice, such as positionally and again by keyword.

Common mistakes to avoid

- State the topic's rule before applying it.
- Trace the data or control flow carefully; check the final value and type.

Quick viva and follow-up cues

- Core answer: Use keywords when they improve clarity, especially for optional parameters; keep parameter names accurate.
- Likely follow-up: A parameter cannot receive the same argument twice, such as positionally and again by keyword.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Use keywords when they improve clarity, especially for optional parameters; keep parameter names accurate.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 21 / 48 | Function Arguments and Command Line Arguments CONCEPT AND DEFINITIONS

*args and **kwargs

Simple definition

`*args` collects extra positional arguments into a tuple. `**kwargs` collects extra named arguments into a dictionary.

Formal / technical definition

In a function definition, starred parameters perform argument packing: `*args` binds surplus positional arguments, and `**kwargs` binds surplus keyword arguments. In a call, the same syntax can unpack an iterable or mapping.

Notation and how to read it

definition: *args -> tuple; **kwargs -> dict

call: *sequence and **mapping unpack values

How it works

- Separate fixed parameters from extra positional and named values.
- Trace packing into a tuple/dictionary or unpacking at the call site.
- Use the collected values without assuming a particular count.

Quick map / trace / syntax

```
extra positional values -> args tuple
extra keyword values -> kwargs dictionary
```

QUICK RECALL

Distinguish packing in a function definition from unpacking at a call site.

TOPIC 21 / 48 | Function Arguments and Command Line Arguments CODE EXAMPLE AND EXAM HELP

*args and **kwargs

Worked code example and output

```
Code:
def show(first, *args, **kwargs):
print(first, args, kwargs)
show(1, 2, 3, color="blue")
Output:
1 (2, 3) {'color': 'blue'}
```

Useful distinction / limitation

Use these when a flexible interface is useful; fixed parameters are easier to read and document when the shape is known.

Common mistakes to avoid

- In a definition, stars pack values; in a call, stars unpack values.
- args is a tuple and kwargs is a dictionary.

Quick viva and follow-up cues

- Core answer: Distinguish packing in a function definition from unpacking at a call site.
- Likely follow-up: Use these when a flexible interface is useful; fixed parameters are easier to read and document when the shape is known.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Distinguish packing in a function definition from unpacking at a call site.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

Command Line Arguments

Simple definition

Command-line arguments are values typed after a script name when the program is launched from a terminal.

Formal / technical definition

The `sys.argv` list exposes command-line tokens: index 0 is the script name, and following elements are argument strings. Programs should validate count and convert values before use.

Notation and how to read it

sys.argv[0] = script path/name

sys.argv[1:] = supplied arguments (all strings)

How it works

- Identify the script name and supplied tokens.
- Check argument count before indexing and convert values if needed.
- Handle invalid or missing arguments with a useful response.

Quick map / trace / syntax

```
terminal command -> interpreter -> sys.argv list -> validation -> program
```

QUICK RECALL

A command-line token is initially text; do not assume the shell or `sys.argv` converts it to a number.

Command Line Arguments

Worked code example and output

```
Code (save as greet.py):
import sys
name = sys.argv[1] if len(sys.argv) > 1 else "friend"
print("Hello,", name)
Run: python greet.py Asha
Output:
Hello, Asha
```

Useful distinction / limitation

Check the argument count before indexing. For numeric arguments, convert and handle conversion errors.

Common mistakes to avoid

- sys.argv values are strings and index 0 is the script name.
- Check length before accessing sys.argv[1].

Quick viva and follow-up cues

- Core answer: A command-line token is initially text; do not assume the shell or `sys.argv` converts it to a number.
- Likely follow-up: Check the argument count before indexing. For numeric arguments, convert and handle conversion errors.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: A command-line token is initially text; do not assume the shell or `sys.argv` converts it to a number.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

MODULE-I COMPLETE

Pause. Review. Then continue.

Python foundations, control flow, exceptions, and function/command-line arguments.

Before you begin the next module

- Close the notes and explain one important idea from memory.
- Redo one code trace without viewing its output.
- Use the next roadmap to preview the order and the ideas you will need.

NEXT MODULE STARTS NEXT

MODULE-II - Functions and Strings

QUICK RECALL

Small regular practice is more useful than memorizing code without tracing it.

MODULE-II - what to study and in what order

Module goal

Learn reusable functions and the full set of syllabus string operations and formatting.

Syllabus map - exact topic order

- Functions: Built-In Functions; Commonly Used Modules; Function Definition and Calling the Function; The return Statement and void Function; Scope and Lifetime of Variables.
- Strings: Creating and Storing Strings; Basic String Operations; Accessing Characters in String by Index Number; String Slicing and Joining; String Methods; Formatting Strings.

Recommended study order

- Learn built-ins and imports before defining and calling your own functions.
- Clarify return values, scope, and lifetime; then study strings as immutable sequences.
- Finish with indexing, slicing, methods, joining, and readable formatting.

QUICK RECALL

Before moving on, explain a return value, locate a variable's scope, and slice a string by hand.

TOPIC 23 / 48 | Functions

CONCEPT AND DEFINITIONS

Built-In Functions

Simple definition

Built-in functions are ready to use without importing a module. They handle common tasks such as counting, conversion, and iteration.

Formal / technical definition

Python's built-in namespace provides functions such as `len`, `range`, `sum`, `min`, `max`, `sorted`, `enumerate`, `zip`, `any`, `all`, and constructors including `int` and `list`.

Notation and how to read it

len(x) -> item count

sum(numbers) -> total

sorted(x) -> new sorted list

How it works

- Identify the input and the function's expected type/shape.
- Call the function and inspect whether it mutates, returns, or iterates.
- Use the result in the next expression or print it.

Quick map / trace / syntax

```
built-in name -> call with arguments -> returned value or action
```

QUICK RECALL

Check whether a function mutates its input or returns a new value, and avoid using a built-in name as your variable name.

TOPIC 23 / 48 | Functions

CODE EXAMPLE AND EXAM HELP

Built-In Functions

Worked code example and output

```
Code:
values = [4, 1, 7]
print(len(values), sum(values), min(values))
print(sorted(values), list(enumerate(values)))
Output:
3 12 1
[1, 4, 7] [(0, 4), (1, 1), (2, 7)]
```

Useful distinction / limitation

`sort()` changes a list in place and returns None; `sorted()` returns a new sorted list.

Common mistakes to avoid

- State the topic's rule before applying it.
- Trace the data or control flow carefully; check the final value and type.

Quick viva and follow-up cues

- Core answer: Check whether a function mutates its input or returns a new value, and avoid using a built-in name as your variable name.
- Likely follow-up: `sort()` changes a list in place and returns None; `sorted()` returns a new sorted list.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Check whether a function mutates its input or returns a new value, and avoid using a built-in name as your variable name.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 24 / 48 | Functions CONCEPT AND DEFINITIONS

Commonly Used Modules

Simple definition

A module is a file or library of reusable Python definitions. Import one when you need functionality beyond the built-ins.

Formal / technical definition

An `import` statement binds a module object or selected names into the current namespace. The standard library includes modules for mathematics, dates, randomness, statistics, file paths, and more.

Notation and how to read it

import module

module.name(...)

from module import name

How it works

- Choose a standard module for the needed operation.
- Import it once and access the documented name.
- Check the returned value and any effect such as randomness or file access.

Quick map / trace / syntax

```
source -> import -> module namespace -> call a reusable function
```

QUICK RECALL

Use a module prefix such as `math.sqrt` when it makes the source of a function clear.

TOPIC 24 / 48 | Functions

CODE EXAMPLE AND EXAM HELP

Commonly Used Modules

Worked code example and output

```
Code:
import math
import statistics
print(math.sqrt(81))
print(statistics.mean([2, 4, 6]))
Output:
9.0
4
```

Useful distinction / limitation

Imports are usually placed near the top of a file. Names imported with `from ... import ...` can shadow existing names.

Common mistakes to avoid

- State the topic's rule before applying it.
- Trace the data or control flow carefully; check the final value and type.

Quick viva and follow-up cues

- Core answer: Use a module prefix such as `math.sqrt` when it makes the source of a function clear.
- Likely follow-up: Imports are usually placed near the top of a file. Names imported with `from ... import ...` can shadow existing names.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Use a module prefix such as `math.sqrt` when it makes the source of a function clear.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 25 / 48 | Functions CONCEPT AND DEFINITIONS

Function Definition and Calling the Function

Simple definition

A function is a named reusable block. Define it with `def`, then call its name with parentheses to run it.

Formal / technical definition

A function definition creates a function object and binds it to a name. Calling evaluates arguments, binds parameters, executes the body, and returns a result (explicitly or implicitly as None).

Notation and how to read it

def name(parameters): suite

name(arguments) -> call

How it works

- Define the function name, parameters, and indented body.
- Call it with matching arguments.
- Trace its return value and distinguish return from print.

Quick map / trace / syntax

```
define once -> call with inputs -> body runs -> result returns to caller
```

QUICK RECALL

Separate parameters (definition) from arguments (values supplied by a call).

TOPIC 25 / 48 | Functions

CODE EXAMPLE AND EXAM HELP

Function Definition and Calling the Function

Worked code example and output

```
Code:
def area(width, height):
return width * height
result = area(5, 3)
print(result)
Output:
15
```

Useful distinction / limitation

The function body does not run when Python reaches `def`; it runs when the function is called.

Common mistakes to avoid

- State the topic's rule before applying it.
- Trace the data or control flow carefully; check the final value and type.

Quick viva and follow-up cues

- Core answer: Separate parameters (definition) from arguments (values supplied by a call).
- Likely follow-up: The function body does not run when Python reaches `def`; it runs when the function is called.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Separate parameters (definition) from arguments (values supplied by a call).

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 26 / 48 | Functions CONCEPT AND DEFINITIONS

The return Statement and void Function

Simple definition

`return` sends a result back to the caller and ends that function call. A function with no return value written still returns `None`.

Formal / technical definition

Executing `return expression` evaluates the expression and terminates the current function, yielding that object to the caller. Falling off the end is equivalent to returning None.

Notation and how to read it

return value -> caller

no explicit return -> None

How it works

- Define the function name, parameters, and indented body.
- Call it with matching arguments.
- Trace its return value and distinguish return from print.

Quick map / trace / syntax

```
function body -> return? -> value to caller
no return -> None
```

QUICK RECALL

Explain whether a function returns data, prints output, or both; do not call a Python function truly void in the C sense.

TOPIC 26 / 48 | Functions CODE EXAMPLE AND EXAM HELP

The return Statement and void Function

Worked code example and output

```
Code:
def square(n):
return n * n
def announce():
print("Ready")
print(square(4))
print(announce())
Output:
16
Ready
None
```

Useful distinction / limitation

Printing a value is not the same as returning it. A caller cannot use printed text as the function's return value.

Common mistakes to avoid

- A function that prints may still return None.
- return ends the current function call.

Quick viva and follow-up cues

- Core answer: Explain whether a function returns data, prints output, or both; do not call a Python function truly void in the C sense.
- Likely follow-up: Printing a value is not the same as returning it. A caller cannot use printed text as the function's return value.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Explain whether a function returns data, prints output, or both; do not call a Python function truly void in the C sense.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 27 / 48 | Functions

CONCEPT AND DEFINITIONS

Scope and Lifetime of Variables

Simple definition

Scope tells where a name can be used. Lifetime tells how long the object/name binding remains available.

Formal / technical definition

Python resolves local names using the LEGB search order: Local, Enclosing, Global, Built-in. A local binding normally exists during a function call; global/module bindings generally remain while the module is loaded.

Notation and how to read it

LEGB = Local -> Enclosing -> Global -> Built-in

How it works

- Mark where each name is assigned: local, enclosing, global, or built-in.
- Apply LEGB lookup when the name is read.
- Explain how long the binding/object remains reachable.

Quick map / trace / syntax

```
function local name shadows same-spelled global name inside that function
```

QUICK RECALL

Distinguish reading a global from assigning to one; explain shadowing and lifetime in terms of the scope where the name is bound.

TOPIC 27 / 48 | Functions

CODE EXAMPLE AND EXAM HELP

Scope and Lifetime of Variables

Worked code example and output

```
Code:
message = "global"
def show():
message = "local"
print(message)
show()
print(message)
Output:
local
global
```

Useful distinction / limitation

Assignment inside a function creates a local name by default. `global` and `nonlocal` alter binding targets and should be used deliberately.

Common mistakes to avoid

- Assignment inside a function is local by default.
- A name's scope and the object's lifetime are related but not identical.

Quick viva and follow-up cues

- Core answer: Distinguish reading a global from assigning to one; explain shadowing and lifetime in terms of the scope where the name is bound.
- Likely follow-up: Assignment inside a function creates a local name by default. `global` and `nonlocal` alter binding targets and should be used deliberately.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Distinguish reading a global from assigning to one; explain shadowing and lifetime in terms of the scope where the name is bound.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 28 / 48 | Strings

CONCEPT AND DEFINITIONS

Creating and Storing Strings

Simple definition

A string stores text. Quotes mark its boundaries; Python strings are immutable, so an operation that seems to change text creates a new string.

Formal / technical definition

`str` is an immutable sequence of Unicode code points. String literals may use single, double, or triple quotes; a variable refers to the string object.

Notation and how to read it

'text' or "text" -> str object

triple quotes can span lines

How it works

- Identify the original string and the requested text operation.
- Apply indexing/slicing/method/formatting rules without mutating the original.
- Show the exact resulting string or list of pieces.

Quick map / trace / syntax

```
name -> immutable string object
reassignment -> name refers to another string
```

QUICK RECALL

Use matching quote styles and remember that indexed string characters cannot be assigned directly.

TOPIC 28 / 48 | Strings

CODE EXAMPLE AND EXAM HELP

Creating and Storing Strings

Worked code example and output

Code:

```
first = 'Python'
second = "notes"
message = first + " " + second
print(message)
```

Output:

```
Python notes
```

Useful distinction / limitation

A string method returns a result; it does not modify the original string in place.

Common mistakes to avoid

- State the topic's rule before applying it.
- Trace the data or control flow carefully; check the final value and type.

Quick viva and follow-up cues

- Core answer: Use matching quote styles and remember that indexed string characters cannot be assigned directly.
- Likely follow-up: A string method returns a result; it does not modify the original string in place.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Use matching quote styles and remember that indexed string characters cannot be assigned directly.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 29 / 48 | Strings

CONCEPT AND DEFINITIONS

Basic String Operations

Simple definition

Strings can be joined, repeated, compared, searched, measured, and checked for membership.

Formal / technical definition

String operators include concatenation `+`, repetition `*`, equality/order comparisons, and membership tests `in`/`not in`; `len` returns the number of code points in the sequence.

Notation and how to read it

a + b -> concatenate

s * n -> repeat

substring in s -> membership

How it works

- Identify the original string and the requested text operation.
- Apply indexing/slicing/method/formatting rules without mutating the original.
- Show the exact resulting string or list of pieces.

Quick map / trace / syntax

```
"Py" + "thon" -> "Python"
"ha" * 3 -> "hahaha"
```

QUICK RECALL

String comparisons are case-sensitive unless you normalize case first.

TOPIC 29 / 48 | Strings

CODE EXAMPLE AND EXAM HELP

Basic String Operations

Worked code example and output

```
Code:
word = "py"
text = word.upper() + "!"
print(text, len(text), "Y" in text)
Output:
PY! 3 True
```

Useful distinction / limitation

Repeated concatenation in a large loop can be inefficient; collect pieces and use `str.join` when building many parts.

Common mistakes to avoid

- State the topic's rule before applying it.
- Trace the data or control flow carefully; check the final value and type.

Quick viva and follow-up cues

- Core answer: String comparisons are case-sensitive unless you normalize case first.
- Likely follow-up: Repeated concatenation in a large loop can be inefficient; collect pieces and use `str.join` when building many parts.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: String comparisons are case-sensitive unless you normalize case first.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 30 / 48 | Strings CONCEPT AND DEFINITIONS

Accessing Characters in String by Index Number

Simple definition

An index selects one character. Index 0 is the first character; negative indexes count backward from the end.

Formal / technical definition

Strings support integer indexing. For a string of length n, valid indexes are `0` through `n-1` and `-n` through `-1`; invalid indexes raise `IndexError`.

Notation and how to read it

0 1 2 3 4 5
P Y T H O N
-6 -5 -4 -3 -2 -1

How it works

- Identify the original string and the requested text operation.
- Apply indexing/slicing/method/formatting rules without mutating the original.
- Show the exact resulting string or list of pieces.

Quick map / trace / syntax

```
word[index] -> one-character string; negative index counts from the right
```

QUICK RECALL

Indexing returns one character, not its numeric position or the character code.

TOPIC 30 / 48 | Strings

CODE EXAMPLE AND EXAM HELP

Accessing Characters in String by Index Number

Worked code example and output

```
Code:
word = "PYTHON"
print(word[0], word[2], word[-1])
Output:
P T N
```

Useful distinction / limitation

Check the length before calculating indexes, and remember that an empty string has no valid character index.

Common mistakes to avoid

- Strings are zero-indexed and immutable.
- Empty strings have no valid index.

Quick viva and follow-up cues

- Core answer: Indexing returns one character, not its numeric position or the character code.
- Likely follow-up: Check the length before calculating indexes, and remember that an empty string has no valid character index.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Indexing returns one character, not its numeric position or the character code.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 31 / 48 | Strings

CONCEPT AND DEFINITIONS

String Slicing and Joining

Simple definition

A slice selects a range of characters. `join` combines several strings with a chosen separator.

Formal / technical definition

The slice `s[start:stop:step]` includes start and excludes stop. `separator.join(iterable)` concatenates string elements, placing the separator between them.

Notation and how to read it

slice: start included, stop excluded

join: separator.join(strings)

How it works

- Identify the original string and the requested text operation.
- Apply indexing/slicing/method/formatting rules without mutating the original.
- Show the exact resulting string or list of pieces.

Quick map / trace / syntax

```
s[1:4] -> indexes 1,2,3
s[::-1] -> reversed copy
"-".join(parts) -> joined text
```

QUICK RECALL

Keep the excluded stop index in mind; call `join` on the separator, not on the list.

TOPIC 31 / 48 | Strings

CODE EXAMPLE AND EXAM HELP

String Slicing and Joining

Worked code example and output

```
Code:
word = "PYTHON"
print(word[1:4], word[::-1])
parts = ["formal", "language", "notes"]
print("-".join(parts))
Output:
YTH NOHTYP
formal-language-notes
```

Useful distinction / limitation

Slicing beyond the end is allowed and is clipped; `join` requires every element to be a string.

Common mistakes to avoid

- The slice stop index is excluded.
- join requires string elements and is called on the separator.

Quick viva and follow-up cues

- Core answer: Keep the excluded stop index in mind; call `join` on the separator, not on the list.
- Likely follow-up: Slicing beyond the end is allowed and is clipped; `join` requires every element to be a string.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Keep the excluded stop index in mind; call `join` on the separator, not on the list.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 32 / 48 | Strings

CONCEPT AND DEFINITIONS

String Methods

Simple definition

String methods provide common text operations such as case changes, trimming, replacement, searching, and splitting.

Formal / technical definition

Methods such as `lower`, `upper`, `strip`, `replace`, `split`, `find`, `startswith`, and `endswith` return values derived from an immutable string.

Notation and how to read it

text.strip() -> trimmed string

text.split(sep) -> list of pieces

text.replace(old,new) -> new string

How it works

- Identify the original string and the requested text operation.
- Apply indexing/slicing/method/formatting rules without mutating the original.
- Show the exact resulting string or list of pieces.

Quick map / trace / syntax

```
original string -> method -> new string/list
original remains unchanged
```

QUICK RECALL

Do not expect `strip` to remove arbitrary words; it removes characters from the ends.

TOPIC 32 / 48 | Strings

CODE EXAMPLE AND EXAM HELP

String Methods

Worked code example and output

```
Code:
line = " Python,OS "
clean = line.strip().lower()
print(clean)
print(clean.split(","))
Output:
python,os
['python', 'os']
```

Useful distinction / limitation

`find` returns -1 when text is absent; `index` raises `ValueError`. Choose based on the desired missing-value behavior.

Common mistakes to avoid

- Methods return new strings because strings are immutable.
- strip removes edge characters; it does not remove a word from the middle.

Quick viva and follow-up cues

- Core answer: Do not expect `strip` to remove arbitrary words; it removes characters from the ends.
- Likely follow-up: `find` returns -1 when text is absent; `index` raises `ValueError`. Choose based on the desired missing-value behavior.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Do not expect `strip` to remove arbitrary words; it removes characters from the ends.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 33 / 48 | Strings

CONCEPT AND DEFINITIONS

Formatting Strings

Simple definition

String formatting inserts values into readable text. F-strings are a clear modern way to place expressions inside braces.

Formal / technical definition

An f-string evaluates replacement fields at runtime and formats their values. The `format` method and percent formatting are also available in Python.

Notation and how to read it

f"...{expression:format_spec}..."

{value:.2f} -> two digits after decimal

How it works

- Identify the original string and the requested text operation.
- Apply indexing/slicing/method/formatting rules without mutating the original.
- Show the exact resulting string or list of pieces.

Quick map / trace / syntax

```
value -> conversion/format spec -> inserted text
```

QUICK RECALL

Formatting produces text; it does not change the numeric value stored in the variable.

TOPIC 33 / 48 | Strings

CODE EXAMPLE AND EXAM HELP

Formatting Strings

Worked code example and output

```
Code:
name = "Asha"
score = 91.456
print(f"{name}: {score:.1f}%")
print("{} scored {}".format(name, 92))
Output:
Asha: 91.5%
Asha scored 92
```

Useful distinction / limitation

Braces inside f-strings mark expressions. Escape a literal brace by doubling it: `{{` or `}}`.

Common mistakes to avoid

- A format spec changes displayed text, not the stored numeric value.
- Double braces when an f-string must show a literal brace.

Quick viva and follow-up cues

- Core answer: Formatting produces text; it does not change the numeric value stored in the variable.
- Likely follow-up: Braces inside f-strings mark expressions. Escape a literal brace by doubling it: `{{` or `}}`.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Formatting produces text; it does not change the numeric value stored in the variable.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

MODULE-II COMPLETE

Pause. Review. Then continue.

Function behavior and strings: scope, return values, indexing, slicing, methods, joining, and formatting.

Before you begin the next module

- Close the notes and explain one important idea from memory.
- Redo one code trace without viewing its output.
- Use the next roadmap to preview the order and the ideas you will need.

NEXT MODULE STARTS NEXT

MODULE-III - Lists and Object-Oriented Programming

QUICK RECALL

Small regular practice is more useful than memorizing code without tracing it.

MODULE-III - what to study and in what order

Module goal

Work confidently with mutable lists, then model data and behavior with Python classes and objects.

Syllabus map - exact topic order

- Lists: Creating Lists; Basic List Operations; Indexing and Slicing in Lists; Built-In Functions Used on Lists; List Methods; The del Statement.
- Object-Oriented Programming: Classes and Objects; Creating Classes in Python; Creating Objects in Python; The Constructor Method; Classes with Multiple Objects; Class Attributes versus Data Attributes; Encapsulation; Inheritance; Polymorphism.

Recommended study order

- Create lists and trace mutation, indexing, slicing, functions, methods, and deletion.
- Define a class, create objects, initialize state, and compare shared vs per-object data.
- Finish with encapsulation, inheritance, and polymorphic method calls.

QUICK RECALL

Before moving on, trace a list mutation and explain how two objects keep separate instance data.

TOPIC 34 / 48 | Lists

CONCEPT AND DEFINITIONS

Creating Lists

Simple definition

A list is an ordered, changeable collection. It can hold repeated values and values of different types.

Formal / technical definition

`list` is a mutable sequence type. Lists may be created with brackets, the `list()` constructor, or comprehensions; elements retain their order.

Notation and how to read it

[] -> empty list

[item1, item2] -> list literal

list(iterable) -> new list

How it works

- Write the initial list and its index positions.
- Apply the requested operation in order.
- Show the new value and whether the original list changed.

Quick map / trace / syntax

```
list reference -> ordered element slots
mutable: an element can be replaced after creation
```

QUICK RECALL

Use commas between elements; distinguish a list literal from a tuple or string.

TOPIC 34 / 48 | Lists

CODE EXAMPLE AND EXAM HELP

Creating Lists

Worked code example and output

```
Code:
empty = []
marks = [82, 76, 91]
mixed = list(("Asha", 20, True))
print(marks, mixed)
Output:
[82, 76, 91] ['Asha', 20, True]
```

Useful distinction / limitation

Two names can refer to the same list object. Mutating it through either name changes the shared object.

Common mistakes to avoid

- State the topic's rule before applying it.
- Trace the data or control flow carefully; check the final value and type.

Quick viva and follow-up cues

- Core answer: Use commas between elements; distinguish a list literal from a tuple or string.
- Likely follow-up: Two names can refer to the same list object. Mutating it through either name changes the shared object.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Use commas between elements; distinguish a list literal from a tuple or string.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 35 / 48 | Lists

CONCEPT AND DEFINITIONS

Basic List Operations

Simple definition

Lists support combining, repeating, checking membership, measuring length, and iterating through items.

Formal / technical definition

Sequence operations on lists include concatenation, repetition, membership, iteration, and `len`. Because lists are mutable, item assignment and many methods update the list object.

Notation and how to read it

a + b -> new concatenated list

a * n -> repeated list

x in a -> membership

How it works

- Write the initial list and its index positions.
- Apply the requested operation in order.
- Show the new value and whether the original list changed.

Quick map / trace / syntax

```
list -> iterate in order / query length / test membership / update an item
```

QUICK RECALL

Track whether an operation mutates the list or produces a new list.

TOPIC 35 / 48 | Lists

CODE EXAMPLE AND EXAM HELP

Basic List Operations

Worked code example and output

```
Code:
values = [2, 4]
values += [6]
values[0] = 1
print(values, len(values), 4 in values)
Output:
[1, 4, 6] 3 True
```

Useful distinction / limitation

`b = a` creates another reference, not a copy. Use `a.copy()` or `a[:]` for a shallow copy.

Common mistakes to avoid

- State the topic's rule before applying it.
- Trace the data or control flow carefully; check the final value and type.

Quick viva and follow-up cues

- Core answer: Track whether an operation mutates the list or produces a new list.
- Likely follow-up: `b = a` creates another reference, not a copy. Use `a.copy()` or `a[:]` for a shallow copy.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Track whether an operation mutates the list or produces a new list.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 36 / 48 | Lists

CONCEPT AND DEFINITIONS

Indexing and Slicing in Lists

Simple definition

Indexing selects one item; slicing selects a range and returns a new list containing those references.

Formal / technical definition

Lists use zero-based integer indexes and support `start:stop:step` slices. Indexed assignment mutates the list; slice assignment can replace a range.

Notation and how to read it

index 0 = first; -1 = last

stop is excluded

slice copy = list of selected elements

How it works

- Write the initial list and its index positions.
- Apply the requested operation in order.
- Show the new value and whether the original list changed.

Quick map / trace / syntax

```
10 20 30 40 50
0 1 2 3 4
```

QUICK RECALL

Check bounds for direct indexing; remember that an out-of-range slice is clipped rather than raising IndexError.

TOPIC 36 / 48 | Lists

CODE EXAMPLE AND EXAM HELP

Indexing and Slicing in Lists

Worked code example and output

```
Code:
items = [10, 20, 30, 40, 50]
print(items[1], items[-1])
print(items[1:4:2])
items[0] = 99
print(items)
Output:
20 50
[20, 40]
[99, 20, 30, 40, 50]
```

Useful distinction / limitation

A slice creates a new outer list but does not deep-copy nested objects.

Common mistakes to avoid

- Indexing outside the list raises IndexError; slicing outside bounds is clipped.
- A slice is a shallow copy, not a deep copy.

Quick viva and follow-up cues

- Core answer: Check bounds for direct indexing; remember that an out-of-range slice is clipped rather than raising IndexError.
- Likely follow-up: A slice creates a new outer list but does not deep-copy nested objects.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Check bounds for direct indexing; remember that an out-of-range slice is clipped rather than raising IndexError.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 37 / 48 | Lists

CONCEPT AND DEFINITIONS

Built-In Functions Used on Lists

Simple definition

Built-in functions can count, total, compare, sort, or enumerate list data without writing each loop by hand.

Formal / technical definition

Common functions include `len`, `sum`, `min`, `max`, `sorted`, `any`, `all`, and `enumerate`. Some return a value; `sorted` creates a new list and `enumerate` returns an iterator of index-value pairs.

Notation and how to read it

sum(values), len(values), min(values), max(values)

sorted(values) -> new sorted list

How it works

- Write the initial list and its index positions.
- Apply the requested operation in order.
- Show the new value and whether the original list changed.

Quick map / trace / syntax

```
list -> built-in function -> result
`enumerate` pairs each index with its value
```

QUICK RECALL

Use `enumerate` instead of manually maintaining an index when traversing items and positions together.

TOPIC 37 / 48 | Lists

CODE EXAMPLE AND EXAM HELP

Built-In Functions Used on Lists

Worked code example and output

Code:

```
values = [8, 3, 5]
print(len(values), sum(values), max(values))
print(sorted(values))
print(list(enumerate(values)))
```

Output:

```
3 16 8
[3, 5, 8]
[(0, 8), (1, 3), (2, 5)]
```

Useful distinction / limitation

`sorted` does not change the original. `sum` is intended for numeric data, not arbitrary mixed lists.

Common mistakes to avoid

- State the topic's rule before applying it.
- Trace the data or control flow carefully; check the final value and type.

Quick viva and follow-up cues

- Core answer: Use `enumerate` instead of manually maintaining an index when traversing items and positions together.
- Likely follow-up: `sorted` does not change the original. `sum` is intended for numeric data, not arbitrary mixed lists.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Use `enumerate` instead of manually maintaining an index when traversing items and positions together.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 38 / 48 | Lists

CONCEPT AND DEFINITIONS

List Methods

Simple definition

List methods add, remove, insert, sort, or copy items. Many change the original list directly.

Formal / technical definition

Mutable-sequence methods include `append`, `extend`, `insert`, `remove`, `pop`, `clear`, `sort`, `reverse`, and `copy`. In-place mutators commonly return None.

Notation and how to read it

append(x): one item
extend(iterable): many items
pop(i): remove and return
sort(): mutate in place

How it works

- Choose the method that matches the requested change.
- Update the original list or save the returned value as appropriate.
- Show the final list and note any removed return value.

Quick map / trace / syntax

```
append([4,5]) -> nested list
extend([4,5]) -> two added elements
```

QUICK RECALL

Do not write `items = items.sort()`; sort mutates the list and returns None. Use `sorted(items)` for a new result.

TOPIC 38 / 48 | Lists

CODE EXAMPLE AND EXAM HELP

List Methods

Worked code example and output

```
Code:
items = [3, 1]
items.append(2)
items.extend([5, 4])
items.sort()
print(items)
Output:
[1, 2, 3, 4, 5]
```

Useful distinction / limitation

`remove(x)` deletes the first matching value; `pop(i)` removes by position and returns it. Missing values/indexes raise errors.

Common mistakes to avoid

- Most in-place mutators return None.
- append adds one item; extend adds each item from an iterable.

Quick viva and follow-up cues

- Core answer: Do not write `items = items.sort()`; sort mutates the list and returns None. Use `sorted(items)` for a new result.
- Likely follow-up: `remove(x)` deletes the first matching value; `pop(i)` removes by position and returns it. Missing values/indexes raise errors.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Do not write `items = items.sort()`; sort mutates the list and returns None. Use `sorted(items)` for a new result.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 39 / 48 | Lists

CONCEPT AND DEFINITIONS

The del Statement

Simple definition

`del` removes a name, list item, or slice. For lists it changes the existing list by deleting selected positions.

Formal / technical definition

The `del` statement unbinds a target name or deletes a subscription/slice target. Deleting list elements shifts later indexes left; deleting a variable name does not guarantee immediate object destruction if references remain.

Notation and how to read it

del name

del sequence[index]

del sequence[start:stop]

How it works

- Write the list with its current indexes.
- Delete the requested index/name/slice.
- Recalculate positions because later elements shift left.

Quick map / trace / syntax

```
before: [10, 20, 30, 40]
del index 1 -> [10, 30, 40]
del slice 1:3 -> [10]
```

QUICK RECALL

`del` is a statement, not a function; it does not return the removed item (use `pop` when the value is needed).

TOPIC 39 / 48 | Lists

CODE EXAMPLE AND EXAM HELP

The del Statement

Worked code example and output

```
Code:
items = [10, 20, 30, 40]
del items[1]
del items[1:3]
print(items)
Output:
[10]
```

Useful distinction / limitation

After one deletion, positions shift. Avoid using old indexes without recalculating them.

Common mistakes to avoid

- del does not return the removed item; pop does.
- Indexes shift after a deletion.

Quick viva and follow-up cues

- Core answer: `del` is a statement, not a function; it does not return the removed item (use `pop` when the value is needed).
- Likely follow-up: After one deletion, positions shift. Avoid using old indexes without recalculating them.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: `del` is a statement, not a function; it does not return the removed item (use `pop` when the value is needed).

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 40 / 48 | Object-Oriented Programming CONCEPT AND DEFINITIONS

Classes and Objects

Simple definition

A class describes a kind of object; an object is one concrete instance with its own data and access to behavior defined by its class.

Formal / technical definition

A class is a user-defined type. An instance is an object created from that type; attributes store state and methods define behavior associated with the class/instance.

Notation and how to read it

class -> blueprint/type

object = instance of class

method -> behavior; attribute -> data

How it works

- Identify the class, instance, attributes, and methods involved.
- Trace attribute lookup and the method call, including self.
- State which data is per-instance and which behavior is inherited/shared.

Quick map / trace / syntax

```
Lamp class
lamp_a object
lamp_b object
Each instance may keep its own state.
```

QUICK RECALL

Explain the distinction: class defines the type; an object is one instance of that type.

TOPIC 40 / 48 | Object-Oriented Programming CODE EXAMPLE AND EXAM HELP

Classes and Objects

Worked code example and output

Code:

```
class Lamp:
def switch_on(self):
return "light"
lamp = Lamp()
print(lamp.switch_on())
```

Output:

```
light
```

Useful distinction / limitation

A class definition creates a class object; it does not automatically create all the instances that may later use it.

Common mistakes to avoid

- State the topic's rule before applying it.
- Trace the data or control flow carefully; check the final value and type.

Quick viva and follow-up cues

- Core answer: Explain the distinction: class defines the type; an object is one instance of that type.
- Likely follow-up: A class definition creates a class object; it does not automatically create all the instances that may later use it.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Explain the distinction: class defines the type; an object is one instance of that type.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 41 / 48 | Object-Oriented Programming CONCEPT AND DEFINITIONS

Creating Classes in Python

Simple definition

Use `class` to define a new type. Indented functions inside it become methods, and assignments can define class attributes.

Formal / technical definition

A class statement executes a suite in a new namespace and creates a class object. Methods are functions stored in the class namespace and participate in descriptor binding when accessed through instances.

Notation and how to read it

class Name:

attributes and methods

How it works

- Identify the class, instance, attributes, and methods involved.
- Trace attribute lookup and the method call, including self.
- State which data is per-instance and which behavior is inherited/shared.

Quick map / trace / syntax

```
class suite -> class namespace -> class object bound to Name
```

QUICK RECALL

The class suite is indented, and method definitions still require `self` when they operate on an instance.

TOPIC 41 / 48 | Object-Oriented Programming

CODE EXAMPLE AND EXAM HELP

Creating Classes in Python

Worked code example and output

```
Code:
class Counter:
kind = "demo"
def value(self):
return 0
print(Counter.kind)
Output:
demo
```

Useful distinction / limitation

Use `self` as the conventional name for the instance parameter; Python passes the instance when a bound method is called.

Common mistakes to avoid

- State the topic's rule before applying it.
- Trace the data or control flow carefully; check the final value and type.

Quick viva and follow-up cues

- Core answer: The class suite is indented, and method definitions still require `self` when they operate on an instance.
- Likely follow-up: Use `self` as the conventional name for the instance parameter; Python passes the instance when a bound method is called.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: The class suite is indented, and method definitions still require `self` when they operate on an instance.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 42 / 48 | Object-Oriented Programming CONCEPT AND DEFINITIONS

Creating Objects in Python

Simple definition

Call a class name like a function to create an object. Then access its attributes and methods with a dot.

Formal / technical definition

Calling a class invokes its construction protocol, normally `__new__` followed by `__init__`, and returns an instance if construction succeeds.

Notation and how to read it

p = Point(2, 5)

p.x -> attribute lookup

p.method() -> bound method call

How it works

- Identify the class, instance, attributes, and methods involved.
- Trace attribute lookup and the method call, including self.
- State which data is per-instance and which behavior is inherited/shared.

Quick map / trace / syntax

```
class call -> new instance -> initialization -> variable refers to object
```

QUICK RECALL

Use the class call to create an instance and dot notation to access its public interface.

TOPIC 42 / 48 | Object-Oriented Programming

CODE EXAMPLE AND EXAM HELP

Creating Objects in Python

Worked code example and output

```
Code:
class Point:
def __init__(self, x, y):
self.x = x
self.y = y
p = Point(2, 5)
print(p.x, p.y)
Output:
2 5
```

Useful distinction / limitation

Two variables can refer to the same object. Reassigning one variable does not automatically copy the object.

Common mistakes to avoid

- State the topic's rule before applying it.
- Trace the data or control flow carefully; check the final value and type.

Quick viva and follow-up cues

- Core answer: Use the class call to create an instance and dot notation to access its public interface.
- Likely follow-up: Two variables can refer to the same object. Reassigning one variable does not automatically copy the object.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Use the class call to create an instance and dot notation to access its public interface.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 43 / 48 | Object-Oriented Programming CONCEPT AND DEFINITIONS

The Constructor Method

Simple definition

`__init__` initializes a new instance after it has been created. It commonly stores the values passed to the class call.

Formal / technical definition

`__init__(self, ...)` is an initializer invoked during normal instance construction. It should initialize instance state and conventionally returns None; it is not the low-level allocator `__new__`.

Notation and how to read it

Student(arguments) -> __init__(self, arguments) initializes that instance

How it works

- Identify the class, instance, attributes, and methods involved.
- Trace attribute lookup and the method call, including self.
- State which data is per-instance and which behavior is inherited/shared.

Quick map / trace / syntax

```
new object -> __init__ assigns starting attributes -> usable object
```

QUICK RECALL

Do not return a non-None value from `__init__`; use it to establish a valid starting state.

TOPIC 43 / 48 | Object-Oriented Programming

CODE EXAMPLE AND EXAM HELP

The Constructor Method

Worked code example and output

```
Code:
class Student:
def __init__(self, name, score=0):
self.name = name
self.score = score
s = Student("Mira", 88)
print(s.name, s.score)
Output:
Mira 88
```

Useful distinction / limitation

Every instance method's first parameter receives the instance. The conventional name is `self`, but it is not a reserved keyword.

Common mistakes to avoid

- __init__ initializes an instance; it is not the allocator __new__.
- Do not return a non-None value from __init__.

Quick viva and follow-up cues

- Core answer: Do not return a non-None value from `__init__`; use it to establish a valid starting state.
- Likely follow-up: Every instance method's first parameter receives the instance. The conventional name is `self`, but it is not a reserved keyword.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Do not return a non-None value from `__init__`; use it to establish a valid starting state.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 44 / 48 | Object-Oriented Programming CONCEPT AND DEFINITIONS

Classes with Multiple Objects

Simple definition

One class can create many objects. Each object can hold different instance data while sharing the same methods.

Formal / technical definition

Each class call normally returns a distinct instance. Instance attributes are stored per object; class-defined methods are shared behavior looked up through the class.

Notation and how to read it

Student(...) -> instance A

Student(...) -> separate instance B

How it works

- Identify the class, instance, attributes, and methods involved.
- Trace attribute lookup and the method call, including self.
- State which data is per-instance and which behavior is inherited/shared.

Quick map / trace / syntax

```
class Student
a -> name='Asha'
b -> name='Ravi'
```

QUICK RECALL

Identify which values belong to each object and which behavior is shared through the class.

TOPIC 44 / 48 | Object-Oriented Programming

CODE EXAMPLE AND EXAM HELP

Classes with Multiple Objects

Worked code example and output

```
Code:
class Student:
def __init__(self, name):
self.name = name
a = Student("Asha")
b = Student("Ravi")
print(a.name, b.name, a is b)
Output:
Asha Ravi False
```

Useful distinction / limitation

If two variables are assigned the same instance, they alias one object; creating the class twice creates separate instances.

Common mistakes to avoid

- State the topic's rule before applying it.
- Trace the data or control flow carefully; check the final value and type.

Quick viva and follow-up cues

- Core answer: Identify which values belong to each object and which behavior is shared through the class.
- Likely follow-up: If two variables are assigned the same instance, they alias one object; creating the class twice creates separate instances.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Identify which values belong to each object and which behavior is shared through the class.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 45 / 48 | Object-Oriented Programming CONCEPT AND DEFINITIONS

Class Attributes versus Data Attributes

Simple definition

A class attribute is stored on the class and can be shared. An instance/data attribute belongs to one particular object.

Formal / technical definition

Attribute lookup on an instance searches the instance namespace and then the class hierarchy. Assigning to `self.name` creates/updates an instance attribute; assigning to `Class.name` changes the class attribute.

Notation and how to read it

Team.sport -> class attribute

a.name / b.name -> instance attributes

How it works

- Identify the class, instance, attributes, and methods involved.
- Trace attribute lookup and the method call, including self.
- State which data is per-instance and which behavior is inherited/shared.

Quick map / trace / syntax

```
class namespace: shared defaults
instance A namespace: its data
instance B namespace: its data
```

QUICK RECALL

Class attributes describe shared class-level data; instance attributes describe each object's individual state.

TOPIC 45 / 48 | Object-Oriented Programming CODE EXAMPLE AND EXAM HELP

Class Attributes versus Data Attributes

Worked code example and output

```
Code:
class Team:
sport = "chess"
def __init__(self, name):
self.name = name
a = Team("Blue")
b = Team("Gold")
print(a.sport, a.name, b.name)
Output:
chess Blue Gold
```

Useful distinction / limitation

A mutable class attribute can be shared and accidentally changed by all instances. Put per-object mutable state on `self`.

Common mistakes to avoid

- Mutable class attributes can be shared unintentionally.
- Store per-object state on self.

Quick viva and follow-up cues

- Core answer: Class attributes describe shared class-level data; instance attributes describe each object's individual state.
- Likely follow-up: A mutable class attribute can be shared and accidentally changed by all instances. Put per-object mutable state on `self`.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Class attributes describe shared class-level data; instance attributes describe each object's individual state.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 46 / 48 | Object-Oriented Programming CONCEPT AND DEFINITIONS

Encapsulation

Simple definition

Encapsulation keeps data and the operations that manage it together, and provides a clear interface for using that data.

Formal / technical definition

Python uses object interfaces, properties, and naming conventions to control intended access. A leading underscore signals non-public use; double-leading underscores trigger name mangling, not strict security.

Notation and how to read it

public operation -> validate input -> update internal state

How it works

- Identify the class, instance, attributes, and methods involved.
- Trace attribute lookup and the method call, including self.
- State which data is per-instance and which behavior is inherited/shared.

Quick map / trace / syntax

```
caller -> deposit(amount) -> account checks rule -> balance changes
```

QUICK RECALL

Explain encapsulation as a protected design boundary and avoid claiming Python private names are impossible to access.

TOPIC 46 / 48 | Object-Oriented Programming CODE EXAMPLE AND EXAM HELP

Encapsulation

Worked code example and output

```
Code:
class BankAccount:
def __init__(self, balance):
self._balance = balance
def deposit(self, amount):
if amount > 0:
self._balance += amount
account = BankAccount(100)
account.deposit(25)
print(account._balance)
Output:
125
```

Useful distinction / limitation

A leading underscore is a convention, not an access-control wall. Keep invariants safe through methods/properties rather than relying on callers to behave.

Common mistakes to avoid

- A leading underscore is a convention, not strict access security.
- Protect invariants through the class interface.

Quick viva and follow-up cues

- Core answer: Explain encapsulation as a protected design boundary and avoid claiming Python private names are impossible to access.
- Likely follow-up: A leading underscore is a convention, not an access-control wall. Keep invariants safe through methods/properties rather than relying on callers to behave.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Explain encapsulation as a protected design boundary and avoid claiming Python private names are impossible to access.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 47 / 48 | Object-Oriented Programming CONCEPT AND DEFINITIONS

Inheritance

Simple definition

Inheritance lets a new class reuse and extend behavior from a parent class. A child class can add methods or override inherited ones.

Formal / technical definition

A derived class lists one or more base classes. Attribute lookup follows the method resolution order; `super()` cooperatively delegates to the next implementation in that order.

Notation and how to read it

class Child(Parent): ...

override -> same method name, new implementation

super() -> next class implementation

How it works

- Identify the class, instance, attributes, and methods involved.
- Trace attribute lookup and the method call, including self.
- State which data is per-instance and which behavior is inherited/shared.

Quick map / trace / syntax

```
Animal -> shared behavior
Dog(Animal) -> inherits and specializes
```

QUICK RECALL

Name the parent-child relationship, inherited behavior, and any overridden method in an exam explanation.

TOPIC 47 / 48 | Object-Oriented Programming CODE EXAMPLE AND EXAM HELP

Inheritance

Worked code example and output

```
Code:
class Animal:
def speak(self):
return "sound"
class Dog(Animal):
def speak(self):
return "woof"
print(Dog().speak())
Output:
woof
```

Useful distinction / limitation

Use inheritance for a genuine 'is-a' relationship. Composition may be clearer when one object merely uses another.

Common mistakes to avoid

- Use inheritance for a real is-a relationship.
- super() follows method resolution order; it does not simply mean 'call the parent' in every complex case.

Quick viva and follow-up cues

- Core answer: Name the parent-child relationship, inherited behavior, and any overridden method in an exam explanation.
- Likely follow-up: Use inheritance for a genuine 'is-a' relationship. Composition may be clearer when one object merely uses another.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Name the parent-child relationship, inherited behavior, and any overridden method in an exam explanation.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 48 / 48 | Object-Oriented Programming CONCEPT AND DEFINITIONS

Polymorphism

Simple definition

Polymorphism means one operation can work with different object types, with each object providing the behavior appropriate to it.

Formal / technical definition

Python supports polymorphic calls through overriding, duck typing, and compatible interfaces. The caller can invoke a method without requiring every object to share one concrete class, provided the needed behavior exists.

Notation and how to read it

same call: object.area()

different runtime object -> its compatible method implementation

How it works

- Identify the class, instance, attributes, and methods involved.
- Trace attribute lookup and the method call, including self.
- State which data is per-instance and which behavior is inherited/shared.

Quick map / trace / syntax

```
loop calls area()
Circle -> circle behavior
Square -> square behavior
```

QUICK RECALL

Describe the common operation and show how different objects respond through their own implementation.

TOPIC 48 / 48 | Object-Oriented Programming CODE EXAMPLE AND EXAM HELP

Polymorphism

Worked code example and output

```
Code:
class Circle:
def area(self): return 3.14
class Square:
def area(self): return 4
for shape in [Circle(), Square()]:
print(shape.area())
Output:
3.14
4
```

Useful distinction / limitation

Duck typing is about supported behavior, not an object's label. If a required method is absent, the call still fails at runtime.

Common mistakes to avoid

- A compatible object still needs to provide the method the caller invokes.
- Polymorphism is not limited to one inheritance tree in Python.

Quick viva and follow-up cues

- Core answer: Describe the common operation and show how different objects respond through their own implementation.
- Likely follow-up: Duck typing is about supported behavior, not an object's label. If a required method is absent, the call still fails at runtime.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Describe the common operation and show how different objects respond through their own implementation.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

Rapid revision: one-line definitions

Python Basics

- Operators: Operators tell Python to perform an action on values, such as add numbers, compare values, or combine conditions.
- Precedence and Associativity: Precedence decides which operator is grouped first.
- Data Types: A data type tells Python what kind of value is stored and which operations make sense for it.
- Indentation: Indentation is the spaces at the start of a line.
- Comments: A comment is a note for people reading code.
- Reading Input: `input()` pauses for a line typed by the user and gives the program that line as text.
- Print Output: `print()` displays values.
- Type Conversions: Type conversion changes a value to another type.
- The type() Function and Is Operator: `type()` reports an object's type.
- Dynamic and Strongly Typed Language: Python is dynamically typed: a name can refer to objects of different types at different times.

Control Flow Statements

- The if Decision Control Flow Statement: An `if` statement runs its indented block only when its condition is true.
- The if...else Decision Control Flow Statement: `if...else` chooses one of two paths: one block runs when the condition is true, the other when it is false.
- The if...elif...else Decision Control Statement: Use `elif` when there are several mutually exclusive cases.
- Nested if Statement: A nested `if` is an `if` inside another `if`.
- The while Loop: A `while` loop repeats a block as long as its condition remains true.
- The for Loop: A `for` loop takes items from an iterable one at a time and runs the block for each item.
- The continue and break Statements: `continue` skips the rest of the current loop turn.
- Catching Exceptions Using try and except Statement: `try` marks code that might fail.

Function Arguments and Command Line Arguments

- Default Parameters: A default parameter supplies a value when the caller leaves that argument out.
- Keyword Arguments: A keyword argument names the parameter in the call, making the purpose clear and allowing arguments to be written in a different order.
- *args and **kwargs: `*args` collects extra positional arguments into a tuple.
- Command Line Arguments: Command-line arguments are values typed after a script name when the program is launched from a terminal.

Rapid revision: one-line definitions

Functions

- Built-In Functions: Built-in functions are ready to use without importing a module.
- Commonly Used Modules: A module is a file or library of reusable Python definitions.
- Function Definition and Calling the Function: A function is a named reusable block.
- The return Statement and void Function: `return` sends a result back to the caller and ends that function call.
- Scope and Lifetime of Variables: Scope tells where a name can be used.

Strings

- Creating and Storing Strings: A string stores text.
- Basic String Operations: Strings can be joined, repeated, compared, searched, measured, and checked for membership.
- Accessing Characters in String by Index Number: An index selects one character.
- String Slicing and Joining: A slice selects a range of characters.
- String Methods: String methods provide common text operations such as case changes, trimming, replacement, searching, and splitting.
- Formatting Strings: String formatting inserts values into readable text.

Rapid revision: one-line definitions

Lists

- Creating Lists: A list is an ordered, changeable collection.
- Basic List Operations: Lists support combining, repeating, checking membership, measuring length, and iterating through items.
- Indexing and Slicing in Lists: Indexing selects one item; slicing selects a range and returns a new list containing those references.
- Built-In Functions Used on Lists: Built-in functions can count, total, compare, sort, or enumerate list data without writing each loop by hand.
- List Methods: List methods add, remove, insert, sort, or copy items.
- The del Statement: `del` removes a name, list item, or slice.

Object-Oriented Programming

- Classes and Objects: A class describes a kind of object; an object is one concrete instance with its own data and access to behavior defined by its class.
- Creating Classes in Python: Use `class` to define a new type.
- Creating Objects in Python: Call a class name like a function to create an object.
- The Constructor Method: `__init__` initializes a new instance after it has been created.
- Classes with Multiple Objects: One class can create many objects.
- Class Attributes versus Data Attributes: A class attribute is stored on the class and can be shared.
- Encapsulation: Encapsulation keeps data and the operations that manage it together, and provides a clear interface for using that data.
- Inheritance: Inheritance lets a new class reuse and extend behavior from a parent class.
- Polymorphism: Polymorphism means one operation can work with different object types, with each object providing the behavior appropriate to it.

Python syntax and exam reference

Operators and decisions

- Arithmetic: +, -, *, /, //, %, **. Comparison: ==, !=, <, <=, >, >=.
- Logical: and/or/not. Membership: in/not in. Identity: is/is not.
- if condition: ...; elif condition: ...; else: ...
- Truth checks: 0, '', [], {}, set(), and None are falsey.

Loops and exceptions

- while condition: ... repeats while true; update state or exit.
- for item in iterable: ... visits yielded items. range(stop) excludes stop.
- continue skips to the next turn; break exits the innermost loop.
- try: risky code / except ValueError: recovery code.

Function arguments

- def f(required, optional=default): ...
- Call by keyword: f(name='Asha'); avoid duplicate parameter binding.
- def f(*args, **kwargs): args is tuple, kwargs is dict.
- sys.argv[0] is script; sys.argv[1:] are text arguments.

Python syntax and exam reference

Function patterns

- def name(parameters): body; call using name(arguments).
- return value sends an object back; no explicit return yields None.
- Built-ins: len, range, sum, min, max, sorted, enumerate, zip.
- LEGB name lookup: Local, Enclosing, Global, Built-in.

String operations

- Strings are immutable sequences. s[i] selects; s[-1] is last.
- s[start:stop:step] includes start and excludes stop.
- separator.join(parts) joins strings; split(separator) returns a list.
- Methods: lower, upper, strip, replace, find, split, startswith.

Formatting cues

- f'{name}: {score:.2f}' inserts values and formats to two decimal places.
- format(value, '.2f') and '{}'.format(value) are alternatives.
- Formatting creates text; it does not change the stored numeric value.
- Use a raw string or escape sequence when backslashes need special treatment.

Python syntax and exam reference

List operations

- Indexing: values[i]; slicing: values[start:stop:step].
- append(x) adds one item; extend(iterable) adds each item.
- pop(i) removes and returns; remove(x) removes first matching value.
- sort() mutates and returns None; sorted(x) returns a new list.
- del values[i] deletes; del values[a:b] deletes a slice.

Class and object cues

- class Name: ... defines a type; Name(...) constructs an instance.
- __init__(self, ...) initializes; self.attribute is per-instance data.
- Class attributes live on the class; mutable ones may be shared.
- Inheritance: class Child(Parent): ...; super() follows MRO.
- Polymorphism: same operation, object-specific compatible behavior.

OOP accuracy checks

- Encapsulation uses interfaces/conventions; underscore is not strict security.
- A method's first parameter receives its instance by convention as self.
- Two objects made separately normally have separate instance attributes.
- Use composition when a 'has-a' relation is clearer than inheritance.

Frequently asked exam and viva themes

MODULE-I - Python basics and control flow

- Explain operators, precedence, types, input/output, and conversions.
- Trace if/elif, nested decisions, while/for loops, and exception handling.
- Show default, keyword, packed arguments, and command-line validation.

MODULE-II - functions and strings

- Define a function and explain arguments, return, scope, and lifetime.
- Use built-ins and modules; distinguish result from side effect.
- Trace indexing, slicing, string methods, joining, and formatting.

MODULE-III - lists and OOP

- Trace list mutation, slicing, methods, and deletion.
- Compare class and instance data; show constructor and multiple objects.
- Explain encapsulation, inheritance, overriding, and polymorphic calls.

How to earn method marks

- Write the rule before the code.
- Show variable values or a trace table.
- State exact output, type, or returned value.
- For OOP, label class, instance, and attributes.

Viva response pattern

- Give a one-sentence definition.
- Name the syntax or rule.
- Show a tiny example.
- Mention one limitation only when relevant.

Final accuracy checklist

- Strings are immutable; list methods may mutate.
- input() and sys.argv values are strings.
- No return statement means None.
- is checks identity; == checks value equality.

Module 1 exam practice and answer cues

Frequently asked long-answer questions

- Explain Python's dynamic and strong typing with examples of reassignment and TypeError.
- Trace a program that uses if/elif, nested decisions, and loops; show the output.
- Compare while and for loops, and demonstrate break and continue.
- Explain positional, keyword, default, *args, and **kwargs argument binding.
- Show how input, conversion, and exception handling work together.
- Write a script that validates command-line arguments and handles invalid input.

Short-answer / viva

- What does input() return?
- Why should `is` not be used for ordinary equality?
- What is the difference between break and continue?
- When is a default parameter evaluated?
- What is stored in sys.argv[0]?
- What does a function return if there is no return statement?

Easy

- Predict 2 + 3 * 4 and explain the grouping.
- Read a name, then print a greeting with print().
- Convert the string '42' to an integer and add one.
- Write a while loop that prints 1, 2, 3.

Moderate

- Grade scores with if/elif/else and test boundary values.
- Use try/except to handle a non-numeric input.
- Write a function with one required and one default parameter.
- Trace a loop containing both continue and break.

Hard

- Write a command-line script that checks argument count, converts a numeric value, and reports errors.
- Design a function that accepts fixed arguments, extra positional values, and named options.
- Explain a case where a mutable default causes state to leak between calls.

Answer cues / final checks

- 2 + 3 * 4 = 14 because multiplication binds before addition.
- input() returns str; int('42') + 1 is 43.
- break exits its nearest loop; continue begins its next iteration.
- No explicit return means None.
- sys.argv[0] is the script; user arguments start at index 1.

Module 2 exam practice and answer cues

Frequently asked long-answer questions

- Define and call a function; compare print and return using a trace.
- Explain built-in functions and modules with short examples.
- Describe scope and lifetime using the LEGB lookup order.
- Explain why strings are immutable and trace indexing and slicing.
- Compare split and join, and show useful string methods.
- Format numeric and text values using f-strings and explain the format spec.

Short-answer / viva

- What is the difference between a parameter and an argument?
- What does a function return by default?
- What does LEGB stand for?
- Why does s.upper() not change s?
- What does a slice stop index mean?
- Which object is used to call join?

Easy

- Write a function that returns the larger of two numbers.
- Extract the first and last character of a non-empty string.
- Join three words with a hyphen.
- Format 12.345 to two decimal places.

Moderate

- Trace a function with a local name that shadows a global name.
- Predict output for negative indexing, a slice, and reversed slicing.
- Normalize a sentence with strip/lower, then split it into words.
- Explain the result of calling a function that only prints.

Hard

- Create a reusable function with keyword-only clarity using the syllabus argument forms and explain bindings.
- Build a formatted report from names and scores; show rounding and alignment.
- Compare `find` and `index` when the target text is absent.

Answer cues / final checks

- A parameter appears in the definition; an argument is supplied by the call.
- LEGB = Local, Enclosing, Global, Built-in.
- Strings are immutable; methods return new values.
- Slices include start and exclude stop.
- separator.join(parts) returns one string.

Module 3 exam practice and answer cues

Frequently asked long-answer questions

- Explain list mutability, aliasing, copying, indexing, and slicing with a trace.
- Compare append, extend, insert, remove, pop, sort, and sorted.
- Explain how del changes a list and why indexes shift.
- Define a class and object; show construction, self, and instance attributes.
- Compare class attributes and instance/data attributes with multiple objects.
- Explain encapsulation, inheritance, and polymorphism with small Python examples.

Short-answer / viva

- How is append different from extend?
- What is a shallow copy?
- Does sort() return a sorted list?
- What does self refer to?
- Are Python underscore names strictly private?
- What is method overriding?

Easy

- Create a list of four numbers and print its length and maximum.
- Read an item by positive and negative index.
- Append one value and extend by two values.
- Create two objects with different names and print each name.

Moderate

- Trace a slice and an indexed assignment, then show the final list.
- Show the difference between pop and remove.
- Use a constructor to initialize two attributes.
- Trace method lookup when a child overrides a parent method.

Hard

- Explain how aliasing changes a list through two variable names; then make a shallow copy.
- Show how a mutable class attribute can be shared and how instance data avoids that bug.
- Design a base class and two child classes with the same method name; explain polymorphic calls.

Answer cues / final checks

- append adds one object; extend iterates over and adds its elements.
- sort mutates and returns None; sorted creates a new list.
- del removes but does not return; pop returns the removed value.
- self is the instance received by an instance method.
- Python's leading underscore is a convention, not enforced secrecy.

Additional / Outside Syllabus: clean code habits

Small habits that make beginner code easier to read

- Choose descriptive names such as total_marks rather than a, b, or x when meaning matters.
- Use consistent indentation and keep one main action per line.
- Break repeated work into a small function with clear parameters and a return value.
- Use f-strings for readable output and comments for intent, not obvious syntax.
- These are general coding practices, added for learning convenience; they are not extra syllabus topics.

A simple trace table

- Before running a loop, create columns for the loop value, changed variables, and printed output.
- Update one row per iteration. This makes off-by-one and accidental reassignment errors visible.
- For lists, add an index column; for objects, record each instance's attributes separately.

QUICK RECALL

Readable code is easier to debug and easier to explain in viva.

Additional / Outside Syllabus: debugging checklist

When output is unexpected

- Read the full error message and locate the line where the exception was raised.
- Check the type and value of each input before changing the algorithm.
- Reduce the program to the smallest example that still shows the problem.
- Trace conditions, loop updates, and function arguments one step at a time.
- For object code, print or inspect the class, instance, and attribute being used.

Example of a useful debugging trace

- Expected: total of [2, 3, 4] is 9.
- Trace: total=0; add 2 -> 2; add 3 -> 5; add 4 -> 9.
- If the result is 7, inspect whether the loop stops too early or skips one item.
- This general debugging method is optional enrichment, not an added syllabus heading.

QUICK RECALL

Find the first step where the actual trace differs from the expected trace.

Created by Anurag Yadav

Email: annuanuragyadav908@gmail.com